

OUROBOROS

ANY TASK → THE LOOP IT NEEDS



BAND OF AGENTS HACKATHON · TRACK 2 — MULTI-AGENT SOFTWARE DEVELOPMENT



any task → the loop it needs

6 AGENTS · ONE ROOM · CROSS-FRAMEWORK · **BAND**-NATIVE · THE LOOP, **ENGINEERED** PER TASK

Stop prompting agents. Start designing the **loop** they run inside.

June 2026 gave it a name — **loop engineering** (Osmani, Cherny). The skill is no longer the perfect prompt; it's the **generate → check → revise** loop with a real exit condition. As one post that hit **6.5M views** put it, eleven days before this deadline:

"You shouldn't be prompting coding agents anymore. You should be designing loops that prompt your agents."

Every coding tool ships **one** loop — and runs it for **everything**.

CSS tweak

Planner → Coder → Reviewer.

DB migration

Planner → Coder → Reviewer. **The identical pipeline.**

A one-line bugfix and an auth change that can leak sessions get the **same** gates, reviewers, and stopping rule. That's vibe coding with extra steps — and building the loop by hand is still expert work nobody's automated.

A band of agents that engineers the loop a task needs — then runs it.

Point it at a task. The room **designs** the verification loop for *that* task — which checks gate it, which critics vote, whether a human must sign — then **runs** it: generate, let the critics attack, run the real checks, revise until a real exit condition holds.

The snake that eats its own tail: the oldest symbol for a loop that feeds itself until the code is whole. What it produces is an artifact you can read — the **LoopSpec** — not just a diff you have to trust.

BUGFIX

Off-by-one in pagination

● compiles

● unit-tests

● repro-test

critics RivalReviewer

max revisions 2 – tight

human gate none

```
checks=[compiles,unit-tests,repro-test] critics=[RivalReviewer] max_rev=2
gate=none
```

FEATURE · HIGH-STAKES

SSO token refresh

● compiles

● unit-tests

● acceptance-test

RivalReviewer

● SecurityCritic – recruited on demand

max revisions 3

human gate 🚫 TechLead must sign

```
checks=[...,acceptance-test] critics=[RivalReviewer,SecurityCritic] max_rev=3
gate=TechLead
```

Touch **auth**, and the room recruits a **SecurityCritic mid-flight** and demands a human signature. The bugfix gets neither. **Engineered for the task – not copy-pasted.**

Take Band out and this is impossible.

① Dynamic recruitment

The room decides *at runtime* it needs a voice nobody added up front, and adds it – `band_add_participant`. Membership grows to fit the task. The most Band-native moment in the system.

② Design + execution, one trail

The loop's *design* and its *execution* live on the **same audit trail** – every handoff an `@mention`. You read the loop a room built *before* you trust the code. The transcript *is* the record.

● ● ● band room · handoff chain

user → `@LoopArchitect` – proposes a loop for the task

`LoopArchitect` → `@LoopCritic` – attacks the exit condition before any code exists

system → `@SecurityCritic` ~ recruited into the room on demand

`LoopRunner` → `@CodeAuthor` → `@QAEngineer` → `@RivalReviewer` – generate · check · judge

system → `@TechLead` 🚫 escalation – human signs before it ships

The checks **really run**. Red is real. Green is earned.

transcript · "Fix off-by-one in pagination offset" · real subprocess QA

user → @LoopArchitect New task: Fix off-by-one in pagination offset – last page drops one row; pure function, no side effects

LoopArchitect → @LoopCritic Proposed a loop for 'Fix off-by-one in pagination offset'. Required gates: compiles, unit-tests, repro-test. Critics: . Human gate: none. – try to poke holes in this exit condition before we build.

LoopCritic → @LoopRunner Loop holds up – exit condition is not gameable. Approved.

LoopRunner → @CodeAuthor loop approved – starting it. draft revision 0; will run the checks and will judge each turn.

CodeAuthor → @QAEEngineer revision 0: first draft. – run the loop's checks on this.

QAEEngineer → @RivalReviewer ran the checks on revision 0 (real subprocess): compiles  unit-tests  repro-test  – 2 check(s) failing. over to you.

 compiles  unit-tests  repro-test

RivalReviewer → @CodeAuthor not signing off (round 0). I broke it:

 compiles  unit-tests  repro-test

CodeAuthor → @QAEEngineer revision 1 [deterministic]: reworked to address 2 defect(s). – run the loop's checks on this.

 compiles  unit-tests  repro-test

QAEEngineer → @RivalReviewer ran the checks on revision 1 (real subprocess): compiles  unit-tests  repro-test  – all checks pass. over to you.

 compiles  unit-tests  repro-test

RivalReviewer → @LoopRunner I tried to break it and couldn't – every required check passes. the exit condition is met; finalize it.

 compiles  unit-tests  repro-test

LoopRunner → @TechLead loop complete: 1 revision(s), gate [n/a], status shipped. shipping code + the loop that produced it.

07 THE REAL BUG, THE REAL FIX, THE REAL FAILURE

REVISION 0 - BUGGY

```
def page(items, page_num, size):  
    start = page_num * size  
    end = start + size - 1  # drops last row  
    return items[start:end]
```

REVISION 1 - SHIPPED

```
def page(items, page_num, size):  
    start = page_num * size  
    end = start + size      # slice end is exclusive  
    return items[start:end]
```

```
repro-test ✖ AssertionError: page returned a short page  
    assert len(page(list(range(10)), 0, 3)) == 3
```

If the loop can't make your tests pass in budget, it stops and hands you a **verified failure** – not a confident wrong answer. The failure mode of agentic coding, closed.

APPLICATION OF TECHNOLOGY Band is load-bearing twice. Dynamic recruitment + a visible design-then-execute handoff chain, all as <code>@mention</code> routing on one trail.	ORIGINALITY The pipeline <i>is</i> the product. Every other entry runs a fixed loop. Ouroboros synthesizes the loop per task – the artifact it ships is a <code>LoopSpec</code>.
BUSINESS VALUE Confident-wrong code, stopped. Loops made inspectable and task-appropriate, with a human gate exactly where the stakes demand one. Auditable by design.	PRESENTATION The thesis fits one screen. Two tasks, two loops, side by side, difference highlighted – then open the transcript and watch it happen live.

Ouroboros

any task → the loop it needs

```
$ python band_repo.py --repo ../your-project \  
  --test "pytest -q" --file src/thing.py --goal "fix the bug"
```

Six cross-framework agents register on Band, coordinate the loop through [@mention](#) handoffs, patch your real file, and only finalize once **your** real tests go green. Verified live.

[Try the live composer →](#)

[Source](#) · [MIT](#)

🐍 built by Natalia Mawyin — because the loop deserves to be engineered, not copy-pasted.